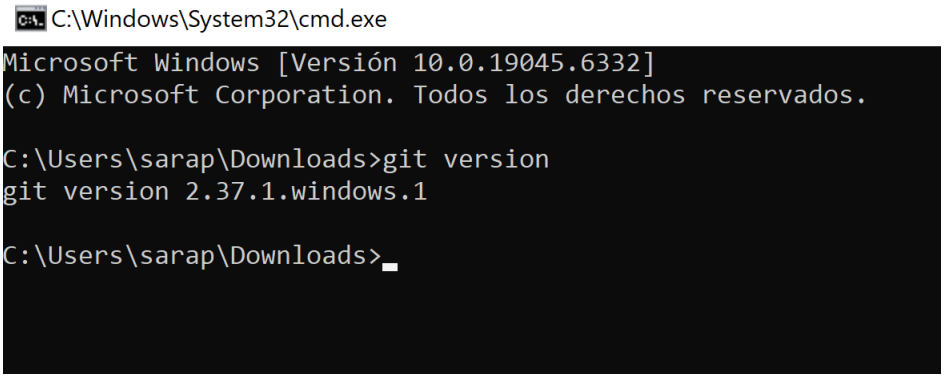
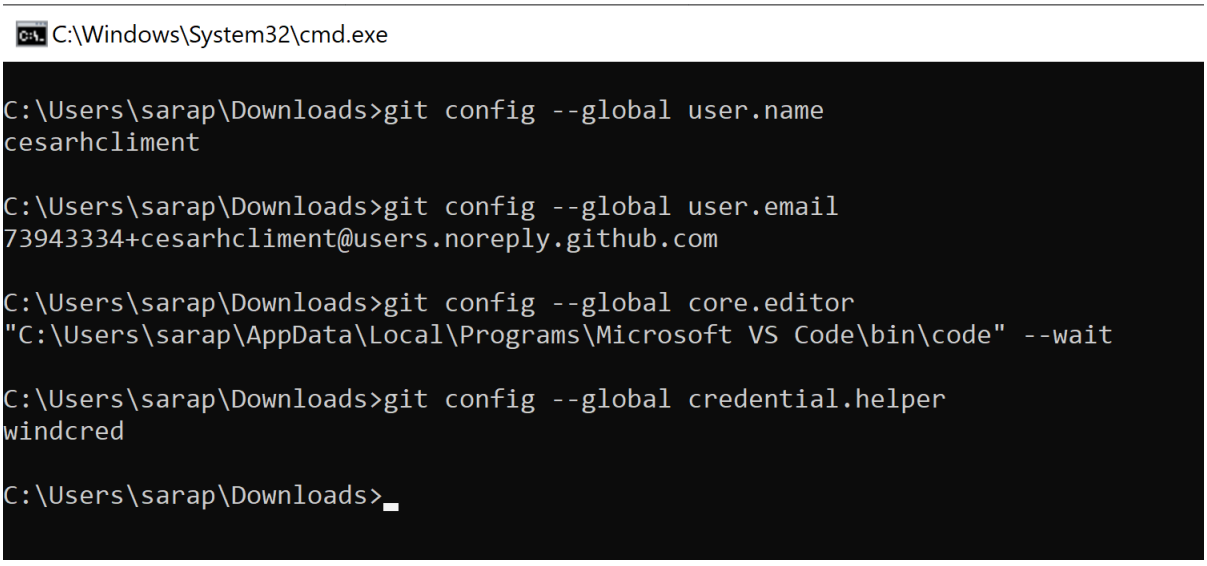
2. Fundamentos de Git I

Las tareas a realizar son:

1. Instala Git en tu sistema operativo. Adjunta una captura de pantalla en la que aparezca el resultado de la ejecución del comando `git --version`.



2. Realiza la **configuración de Git** según lo indicado en el tema (nombre, correo electrónico y editor de preferencia). Adjunta una captura de pantalla con el resultado de la ejecución de los comandos de configuración.



3. Crea una carpeta denominada `S1R1`. Realiza las siguientes acciones en ella:

md S1R1
cd S1R1

1. Crea un repositorio Git.

git init

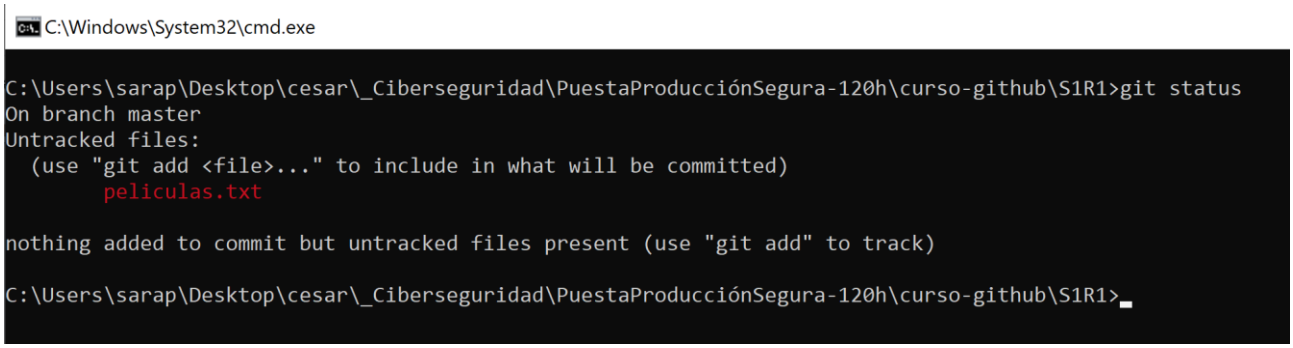
2. Crea un fichero denominado `libros.txt`. Añade tres títulos de libros cada uno en una línea distinta.
3. Haz un primer *commit*.

git status
git add .
git status
git commit –m “commit 1”

4. Añade dos libros al archivo `libros.txt`.
5. Haz un segundo *commit*.

git status
git add .
git status
git commit –m “commit 2”

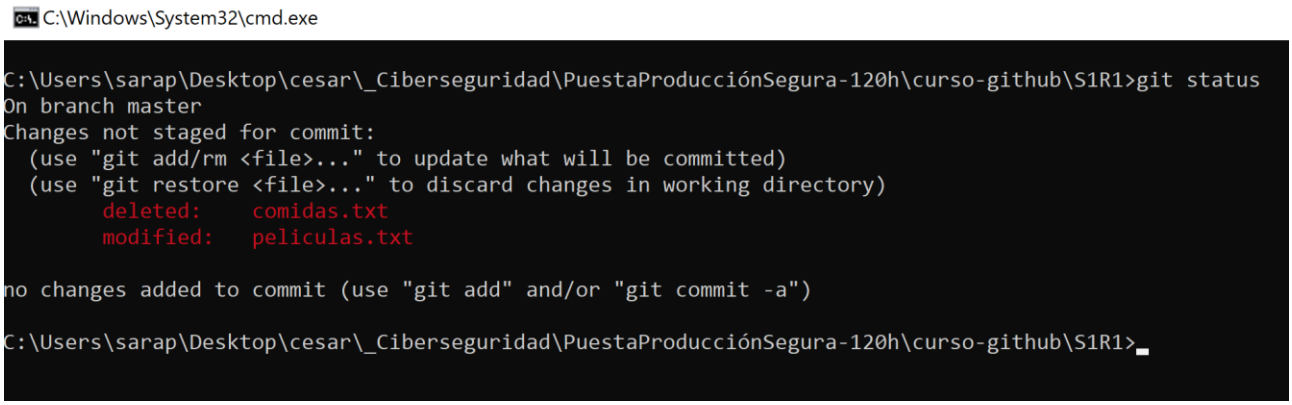
6. Crea un fichero denominado `peliculas.txt`. Añade tres títulos de películas a dicho archivo.
7. Haz una captura de pantalla del comando `git status`.



8. Crea un fichero denominado `comidas.txt`. Añade tres nombres de comidas a dicho archivo.
9. Haz un tercer *commit* que incluya los archivos `peliculas.txt` y `comidas.txt`.

git status
git add .
git status
git commit –m “commit 3”

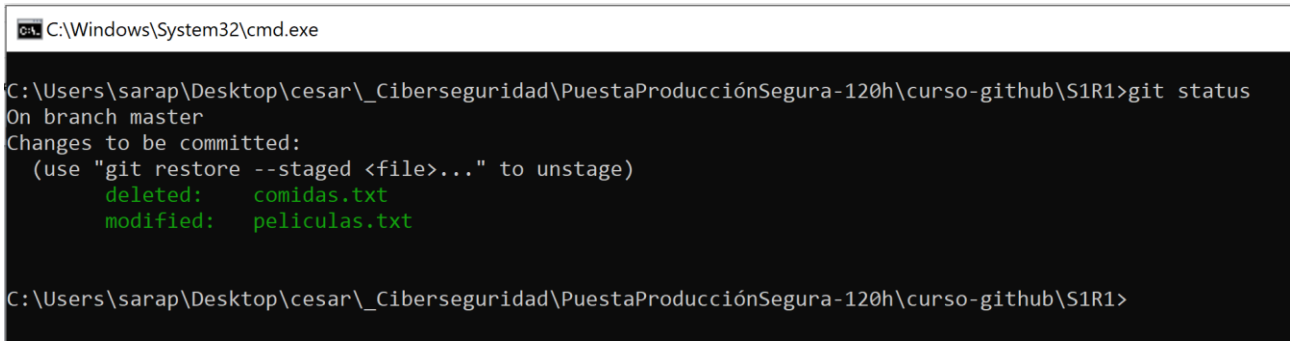
10. Elimina el archivo `comidas.txt` desde el navegador de archivos.
11. Añade dos películas más al archivo `peliculas.txt`.
12. Haz una captura de pantalla que muestre los cambios en el directorio de trabajo.



13. Añade los cambios al área de preparación.

git add .

14. Haz una captura de pantalla del comando `git status`. Debe indicar que se ha borrado el archivo `comidas.txt` y que se ha modificado el archivo `peliculas.txt`.



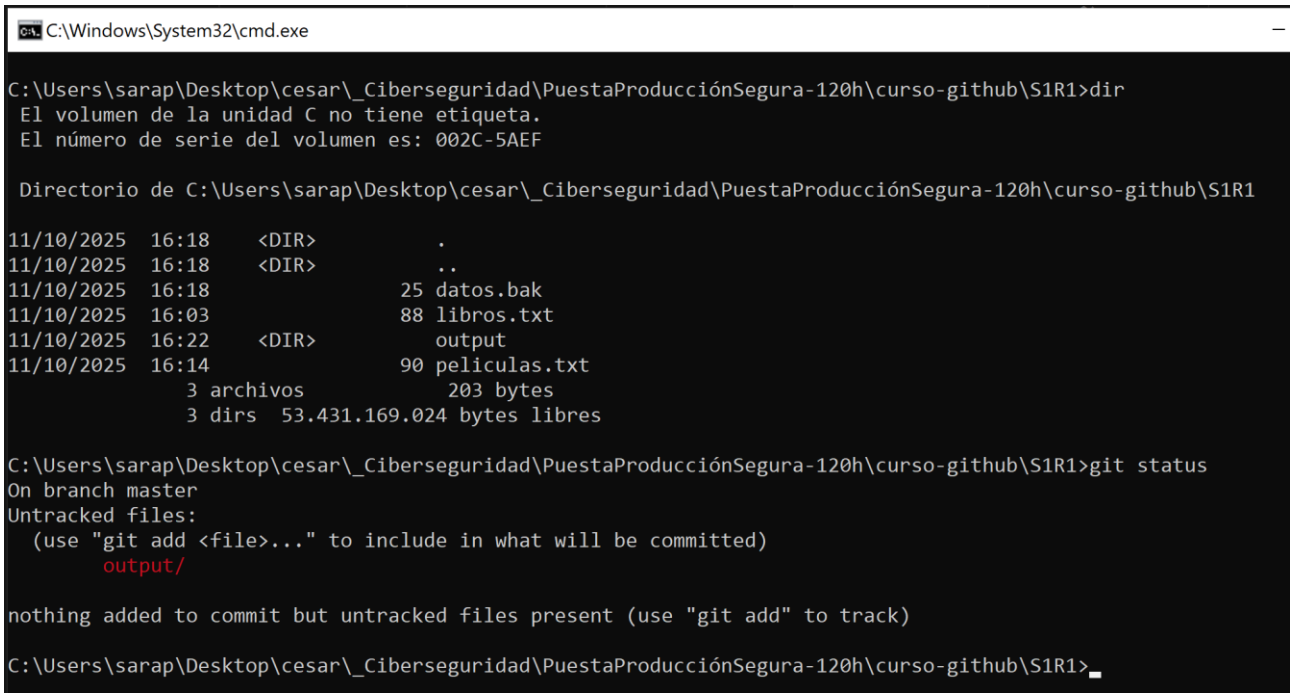
15. Haz un cuarto *commit*.

git commit –m “comit 4”

16. Crea un archivo denominado `datos.bak`. Añade tres títulos de libros a dicho archivo. **¡IMPORTANTE! No añadas el archivo al área de preparación ni hagas ningún commit.**
17. Crea una subcarpeta denominada `output`. Crea un archivo denominado `salida.txt` en su interior. Escribe tu nombre y apellidos en dicho archivo. **¡IMPORTANTE! No añadas los archivos al área de preparación ni hagas ningún commit.**

md output
cd output

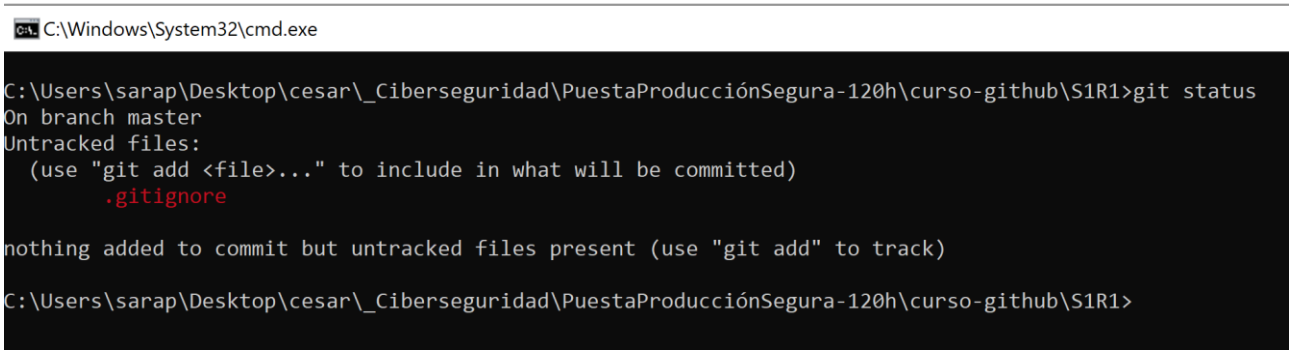
18. Haz una captura de pantalla del comando `git status`. Deben aparecer el archivo `datos.bak` y la carpeta `output` como archivos nuevos (color rojo). Recuerda que, por defecto, `git` no muestra el contenido de una carpeta desconocida, sino solo el nombre de dicha carpeta; si se desea mostrar los archivos nuevos dentro de carpetas desconocidas se debe ejecutar `git status -u`.



19. Crea un archivo `.gitignore` para que los ficheros con extensión `.bak` y el contenido de la carpeta `output/` no se incluyan en el repositorio.

***.bak**
output/

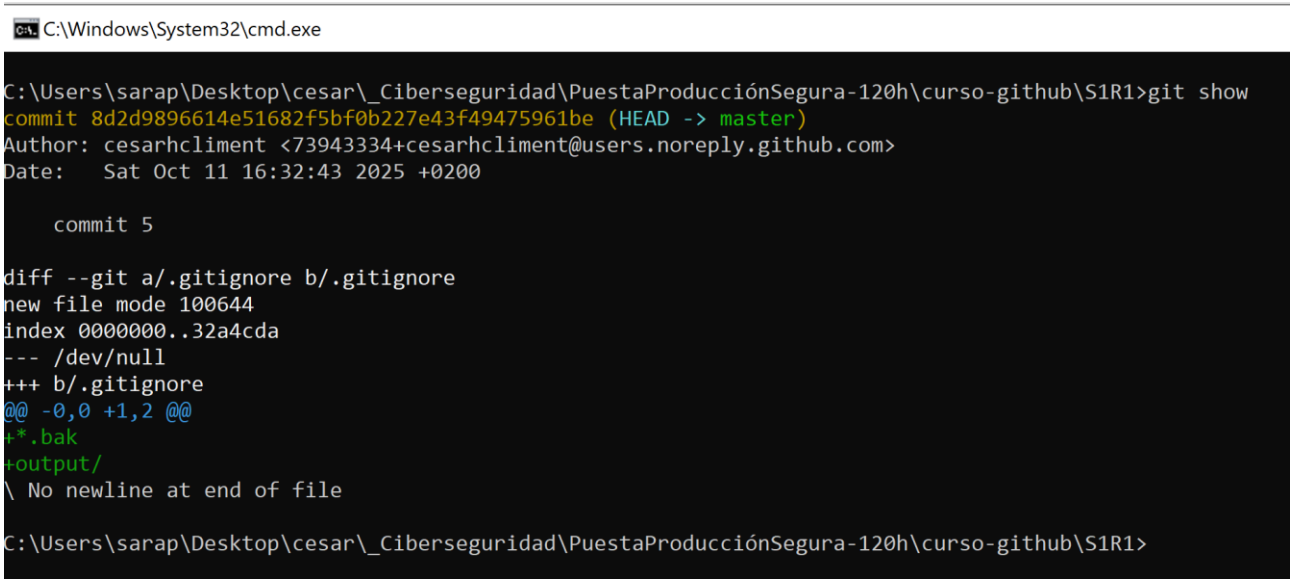
20. Haz una nueva captura de pantalla del comando `git status`. Ahora no deben aparecer los archivos `datos.bak` y `output/salida.txt` como archivos nuevos, sino que en su lugar debe aparecer únicamente el archivo `.gitignore`.



21. Haz un último *commit* para incluir el archivo `.gitignore` en el repositorio.

git add .
git commit –m “comit 5”

22. Haz una captura de pantalla que muestre el histórico de cambios del repositorio.



3. Fundamentos de Git II

1. Crea una carpeta denominada S2R1. Realiza las siguientes acciones en ella:

md S2R1
cd S2R1

a) Crea un repositorio Git.

git init

b) Crea un fichero denominado actores.txt. Añade tres nombres de actores cada uno en una linea distinta.
c) Haz un primer commit.

git add .
git commit –m “commit 1”

d) Crea una rama denominada test.

git branch test

e) Cambia a la rama test

git checkout test

f) En la rama test crea un fichero denominado actrices.txt. Añade tres nombres de actrices y realiza un commit en dicha rama.

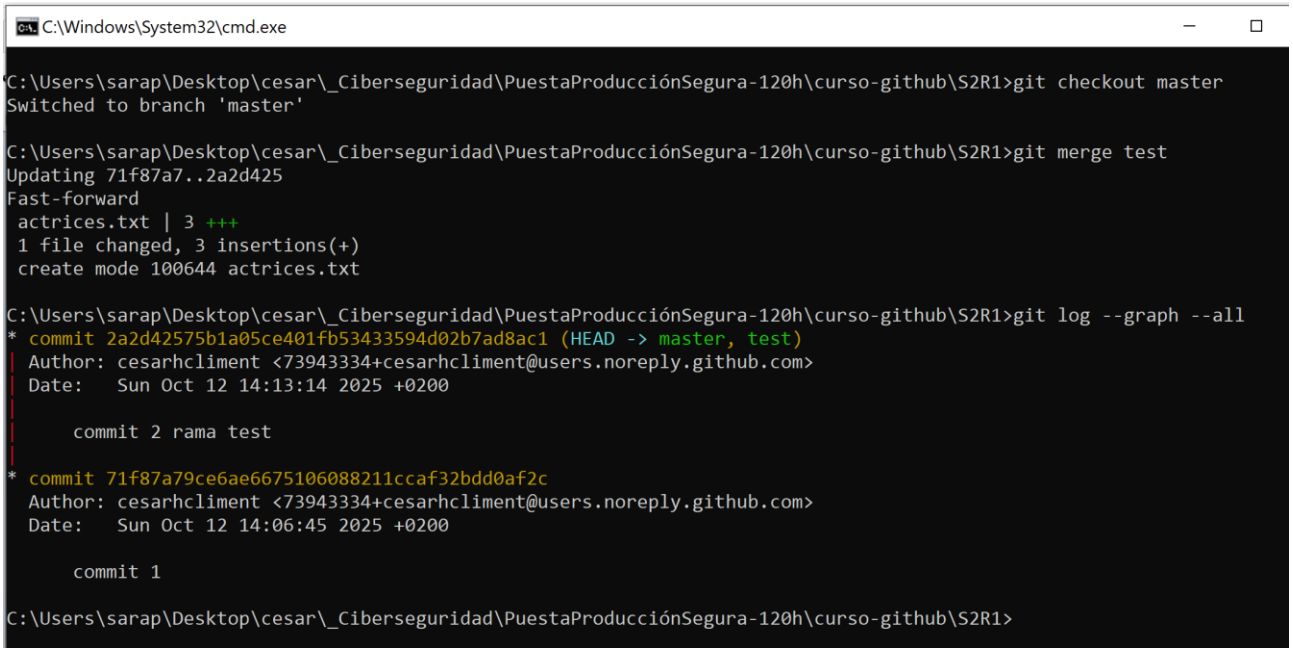
git add .
git commit –m “commit 2 rama test”

g) Haz una captura de pantalla del resultado del comando git log --graph --all.



h) Cambia a la rama master.
i) Incorpora los cambios de la rama test a la rama master. Haz una captura de pantalla de los comandos que has utilizado y de su resultado.

git checkout master
git merge test
git log --graph --all

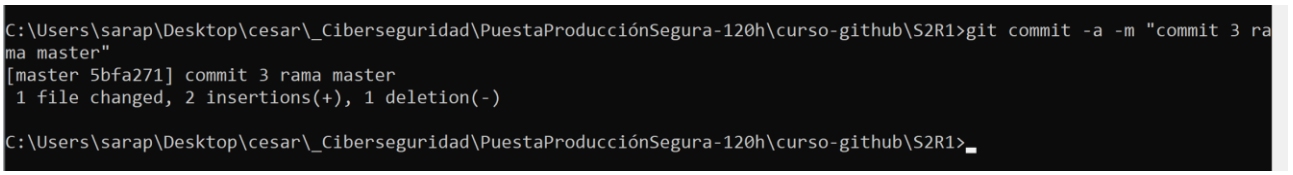


j) Crea una segunda rama denominada test2. La rama test2 apunta al mismo commit que la rama master en este momento.

git branch test2

k) En la rama master, anade una actriz al fichero actrices.txt y haz un commit.

git add .
git commit –m “commit 3 rama master”



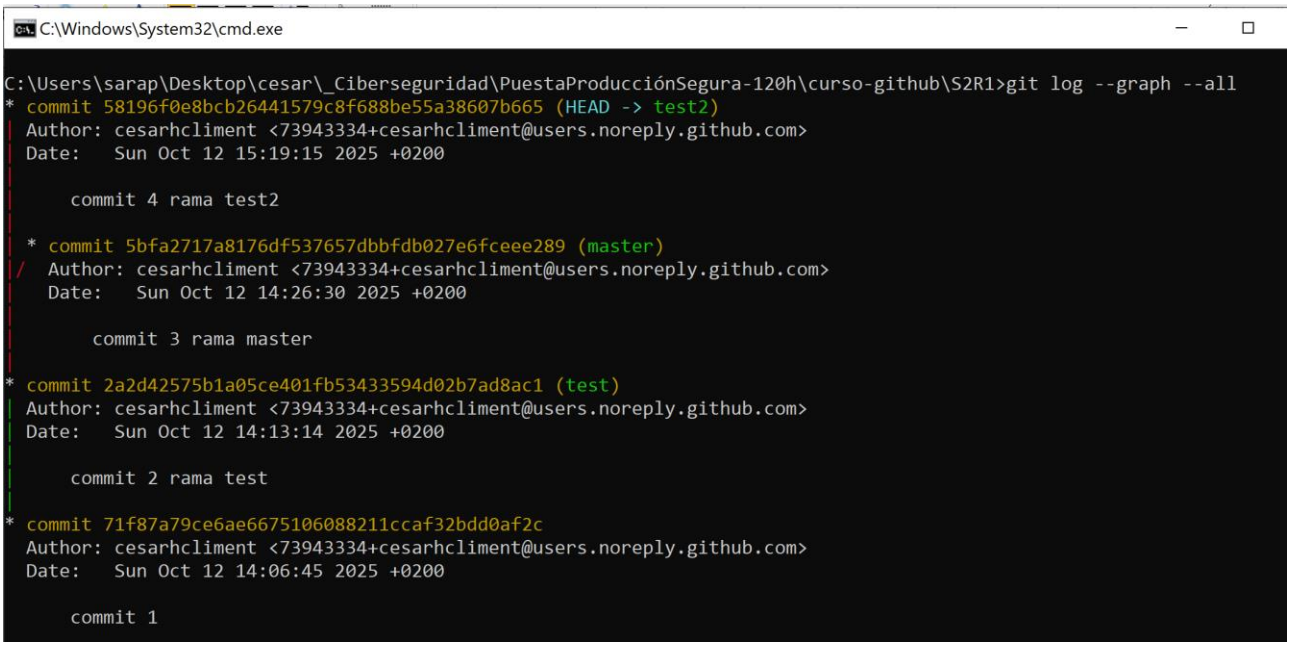
l) Cambia a la rama test2

git checkout test2

m) En la rama test2, anade una actriz al fichero actrices.txt y haz otro commit.

git add .
git commit –m “commit 4 rama test2”

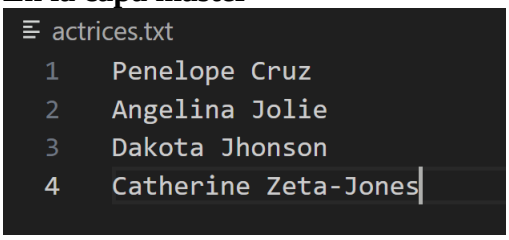
n) Haz una captura de pantalla del resultado del comando git log --graph --all. Debe haber dos caminos distintos: uno para la rama master y otro para la rama test2.



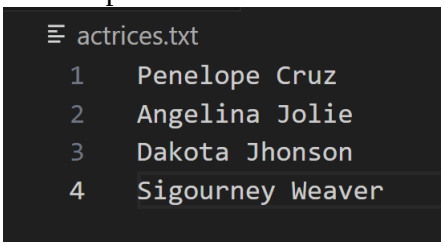
n) Cambia a la rama master

git checkout master

En la capa master

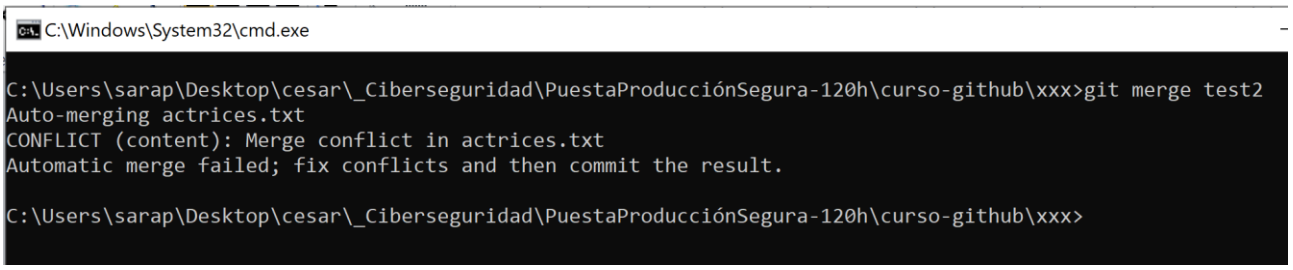


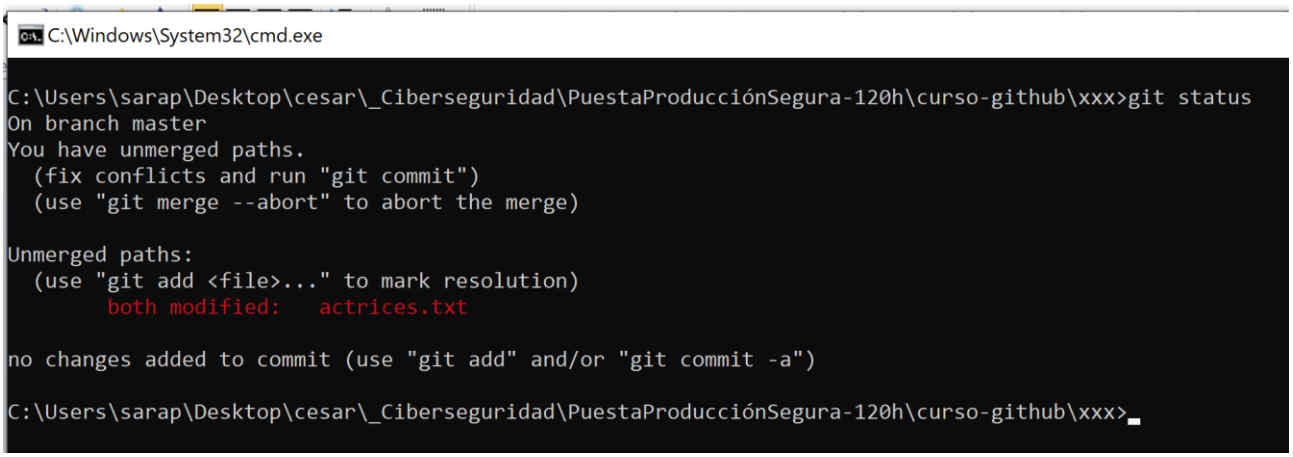
En la capa test2



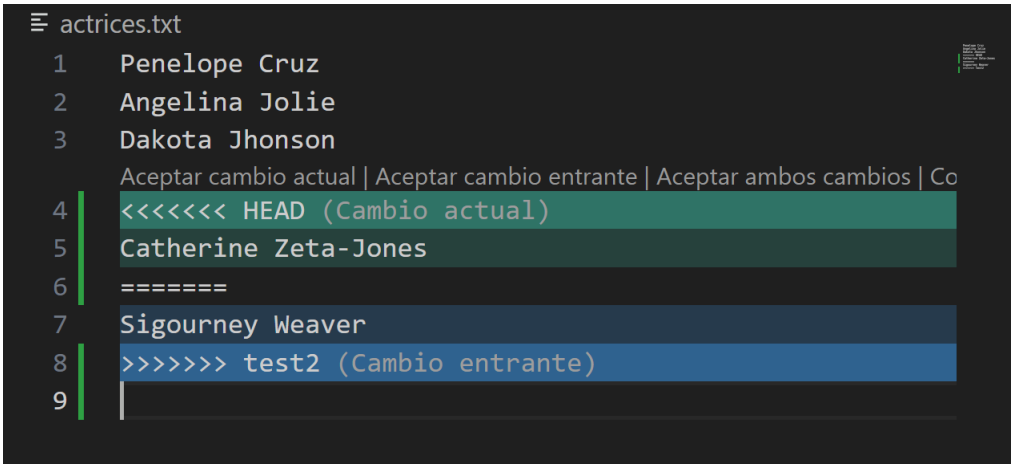
o) Incorpora los cambios de la rama test2 a la rama master. Se produce un conflicto? De ser asi realiza una captura del comando git status.

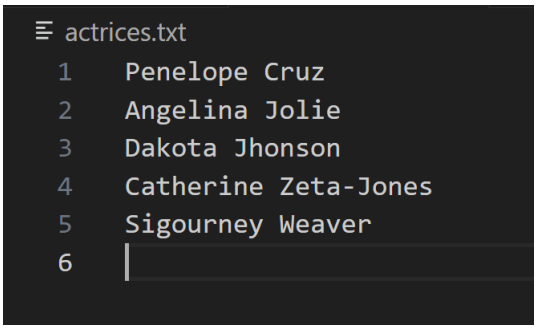
git merge test2





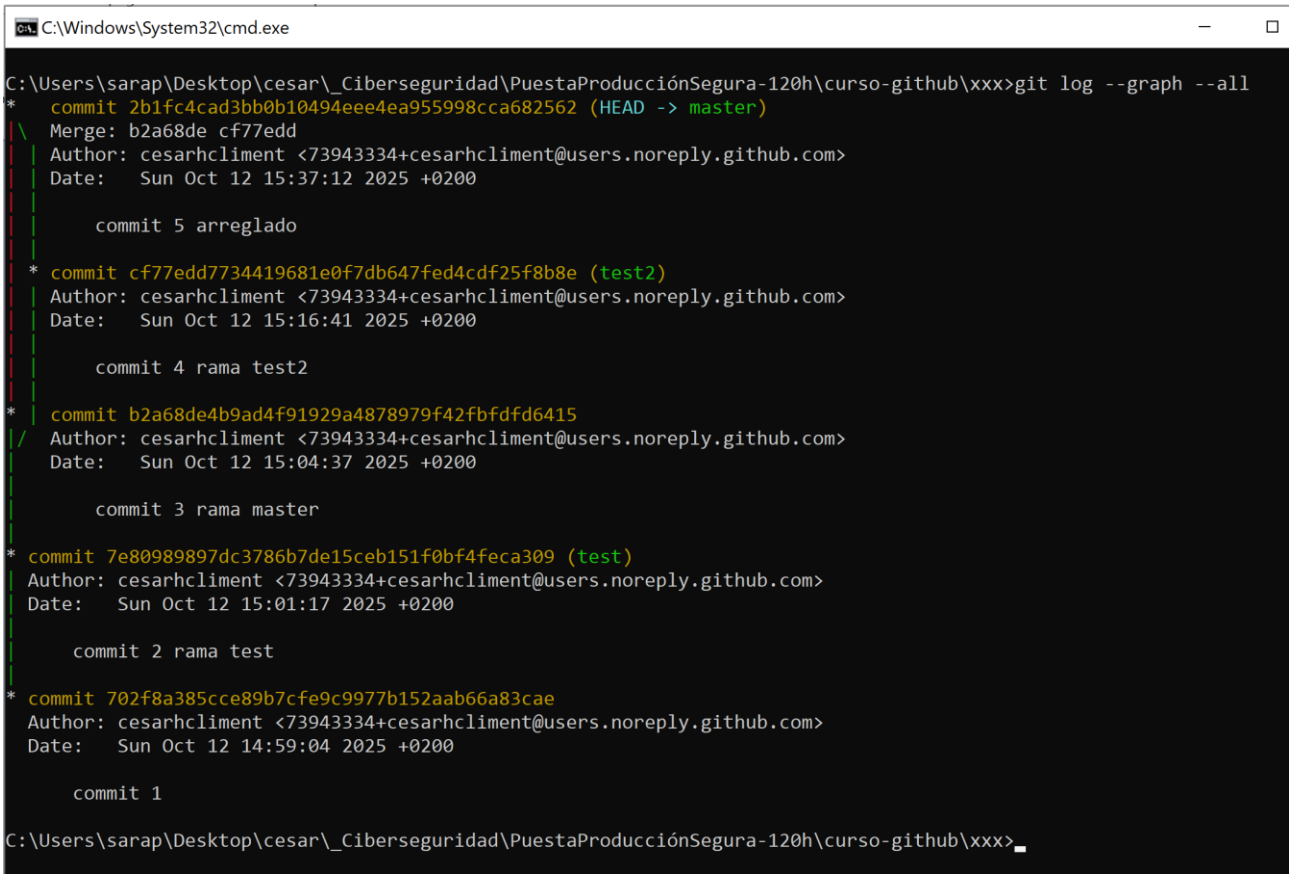
p) Resuelve el conflicto incorporando los dos nombres de actrices.





git add .
git commit –m “commit 5 arreglado”

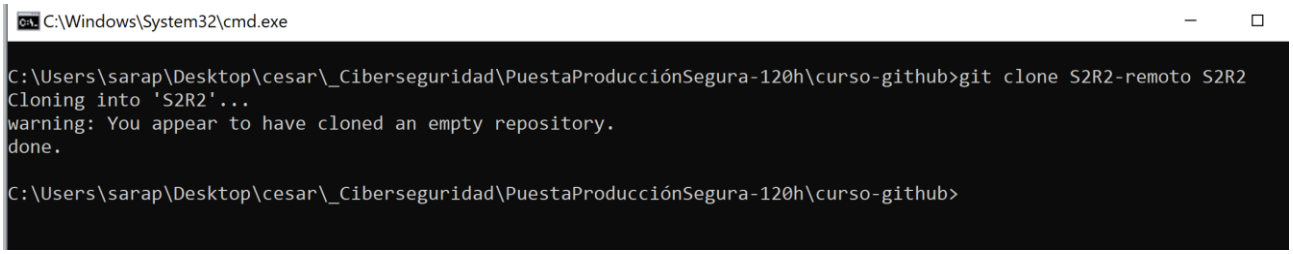
q) Haz una captura de pantalla del resultado del comando git log --graph --all. Observa que se ha creado un nuevo commit que integra los dos caminos anteriores.



2. Crea una carpeta denominada S2R2-remoto. Inicializa un repositorio Git en su interior mediante el comando **git init --bare**. Esta carpeta se utilizara como repositorio remoto.

md S2R2-remoto
cd S2R2-remoto
git init --bare

3. Clona el repositorio S2R2-remoto en una carpeta denominada S2R2.
Adjunta captura de pantalla del resultado del comando de clonado.

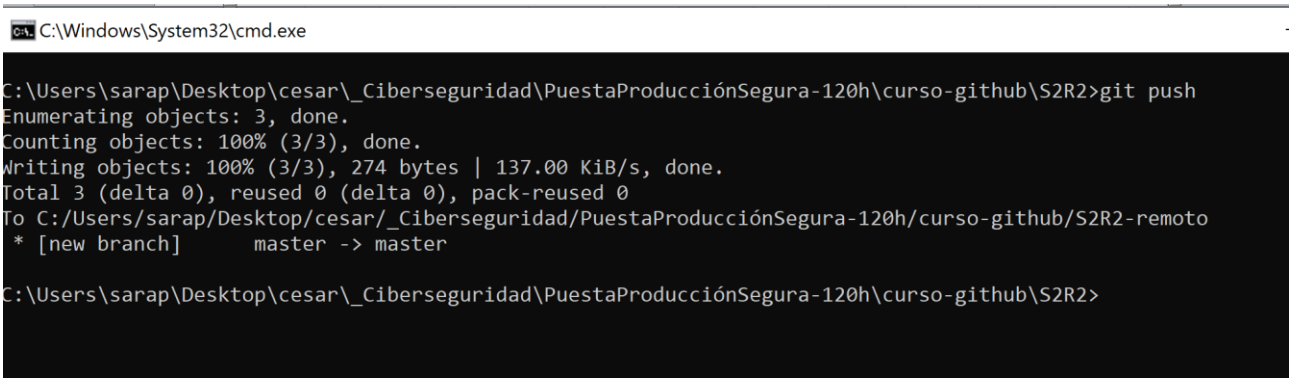


A continuacion realiza las siguientes acciones en el repositorio S2R2:
a) Crea un archivo denominado directores.txt. Anade el nombre de tres directores de cine.
b) Haz un commit.

cd S2R2
git add .
git commit –m “commit 1”

c) Realiza un push al repositorio remoto. Adjunta captura de pantalla del resultado.

git push



d) Crea una rama denominada version1.

git branch version1

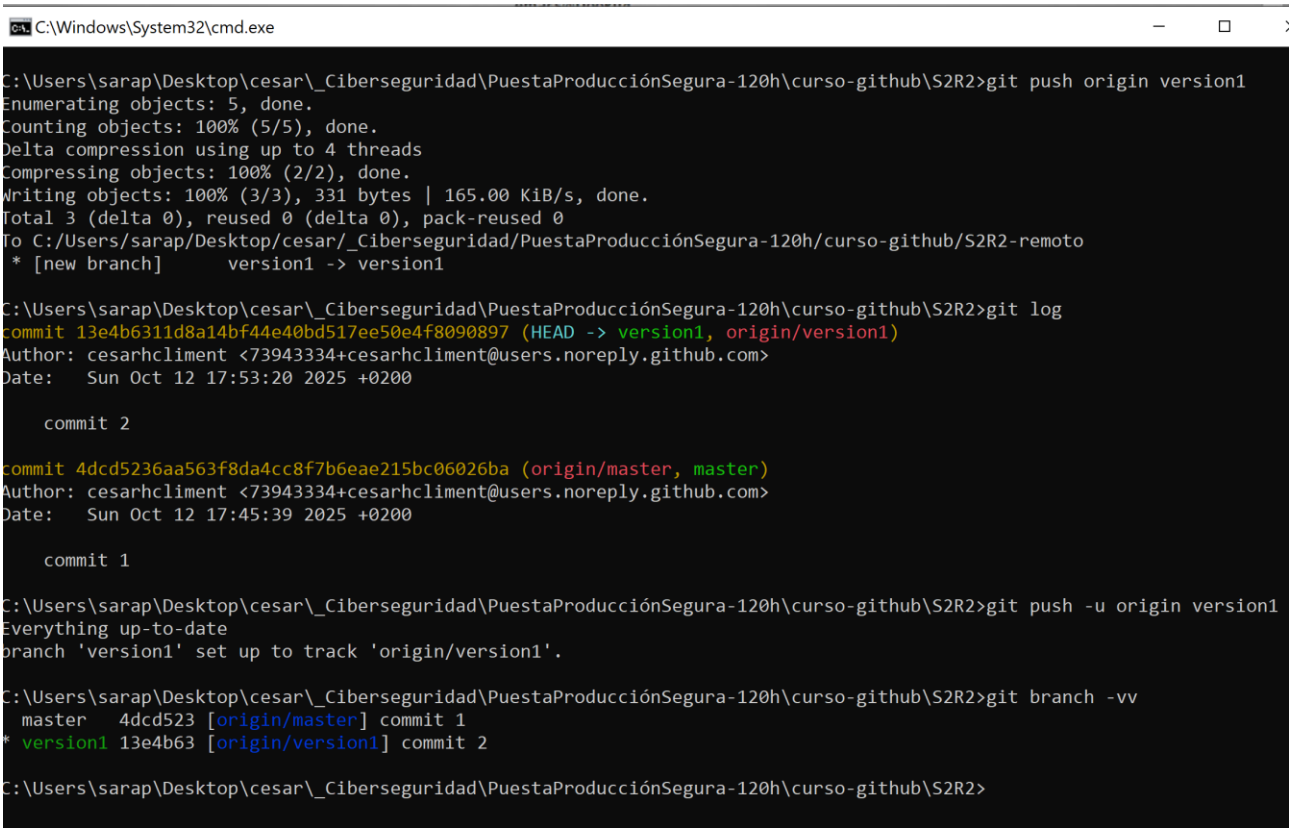
e) Cambia a la rama version1.

git checkout version1

f) En la rama version1 anade el nombre de dos directores de cine mas al archivo directores.txt y haz un commit de los cambios.

git add .
git commit –m “commit 2”

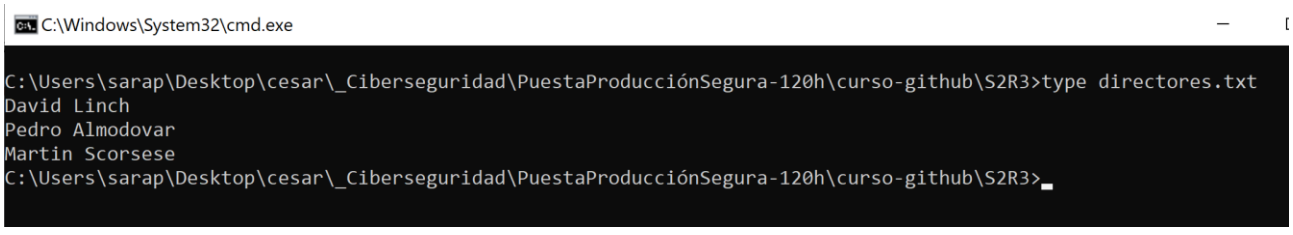
g) Realiza un push de la rama al repositorio remoto de manera que quede asociada a la rama remota del mismo nombre. Adjunta captura de pantalla del resultado.



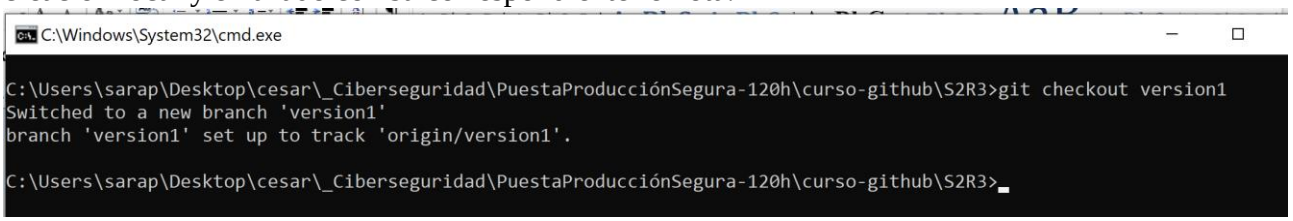
3. Clona el repositorio S2R2-remoto en una segunda carpeta denominada S2R3. Realiza las siguientes acciones sobre ella:

git clone S2R2-remoto S2R3

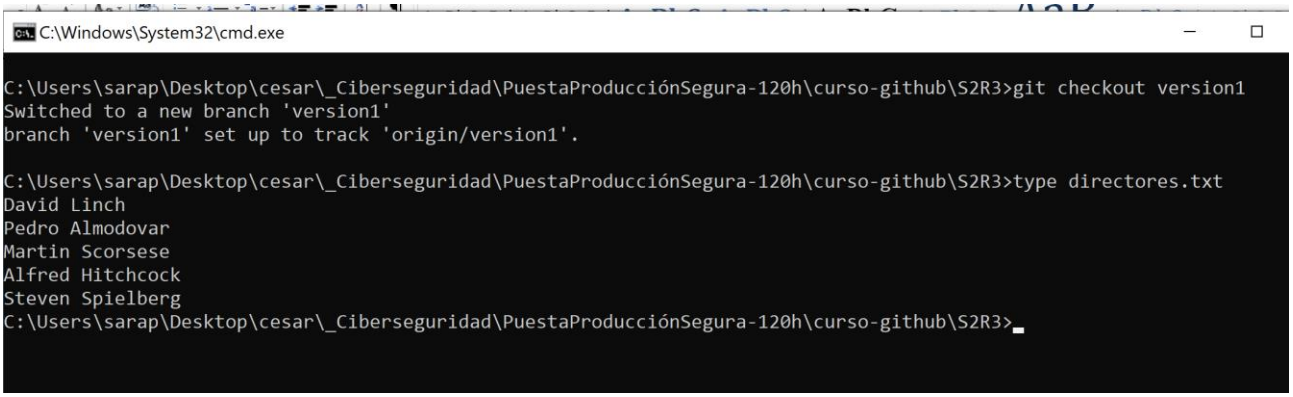
a) Muestra en la consola el contenido del fichero directores.txt y el resultado del comando git status. Debe mostrar tres directores.



b) Cambia a la rama version1. Muestra el resultado del comando.
Comprueba que se crea una rama local version1 con el contenido de la rama remota origin/version1 y enlazada con ella. Al clonar el repositorio la rama no existia (solo se clona la rama principal, master), pero al cambiar a una rama que existe en el remoto se produce su creacion local y enlazado con su correspondiente remota.



c) Muestra el contenido del fichero directores.txt por la pantalla.
Comprueba que se muestran los 5 nombres de directores esperados.
Adjunta captura de pantalla.



d) Cambia a la rama master.

git checkout master

e) Incorpora los cambios de la rama version1 a la rama master.

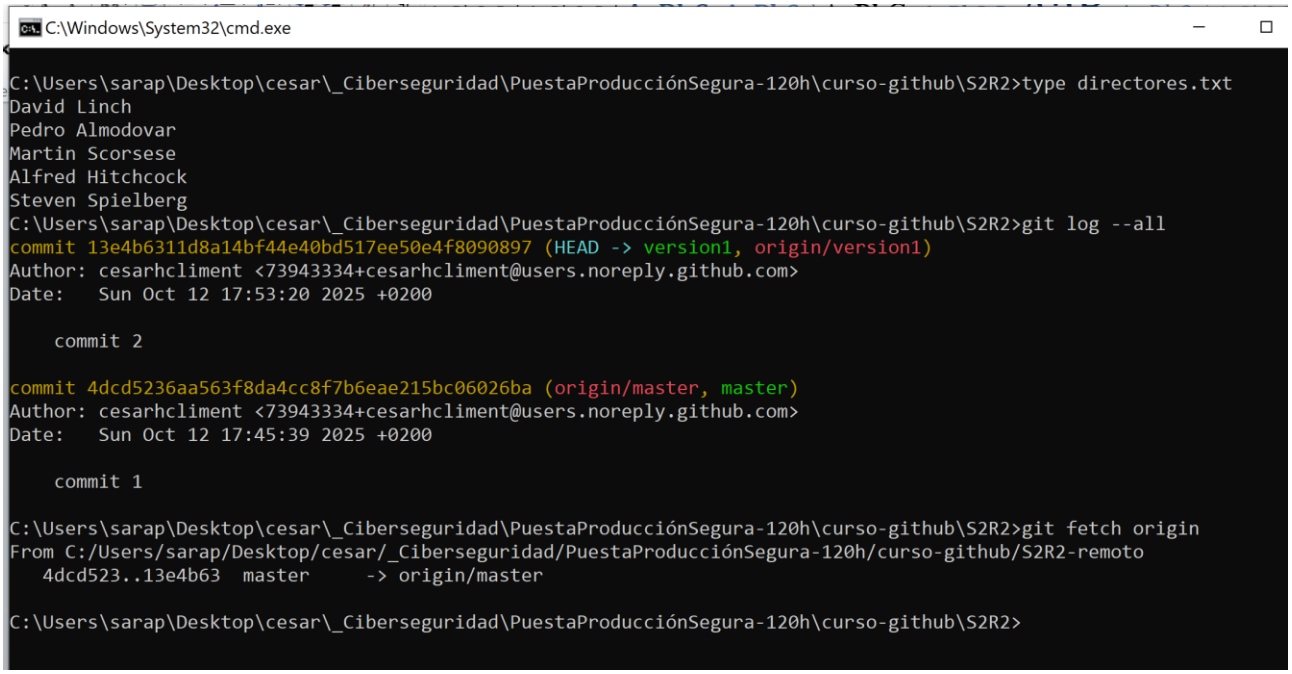
git merge version1

f) Sube la rama master actualizada al servidor. Adjunta captura de pantalla del resultado del comando.

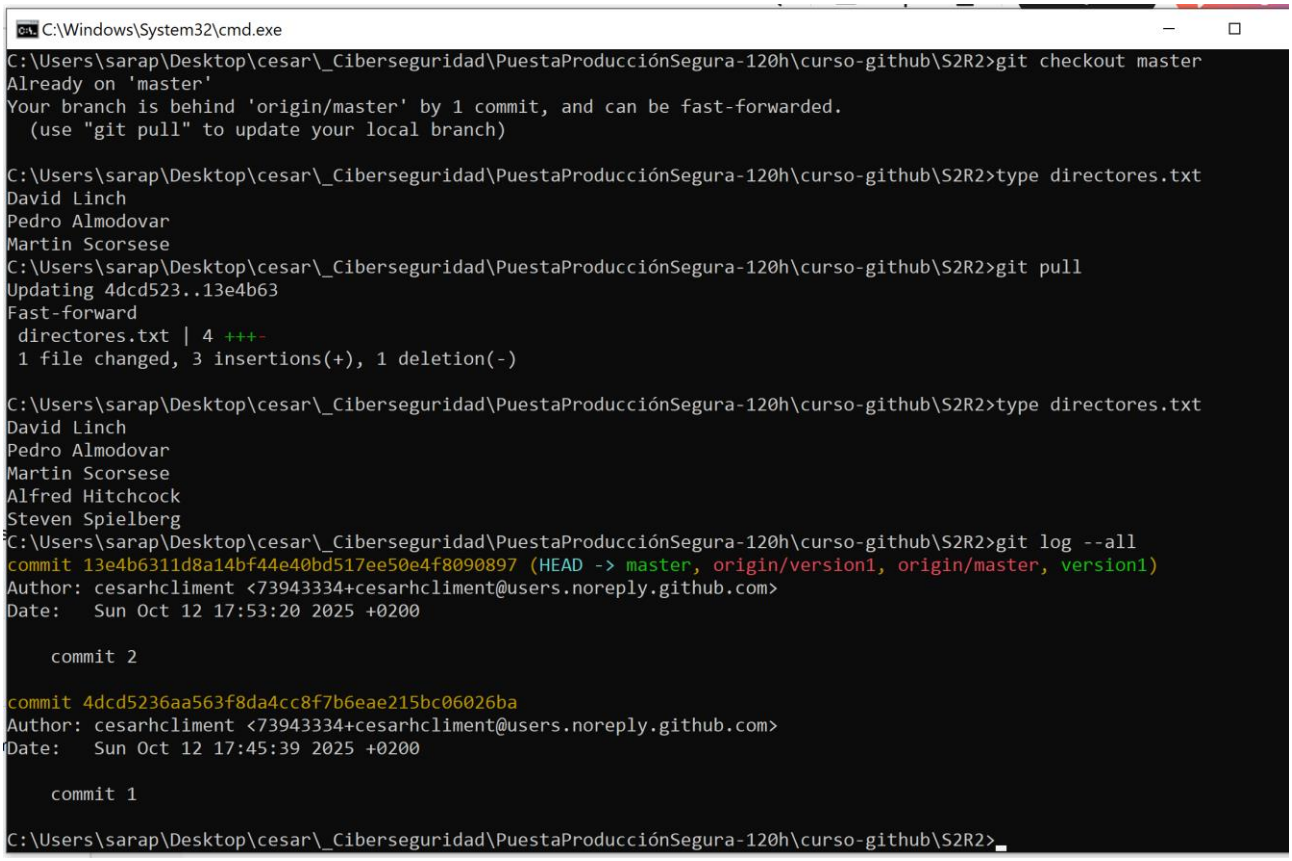


5. Vuelve de nuevo a la carpeta S2R2 y realiza las siguientes acciones:
a) Obten los cambios que hay en el repositorio remoto sin fusionarlos en la rama local. Adjunta captura de pantalla del resultado del comando utilizado.

git fetch origin

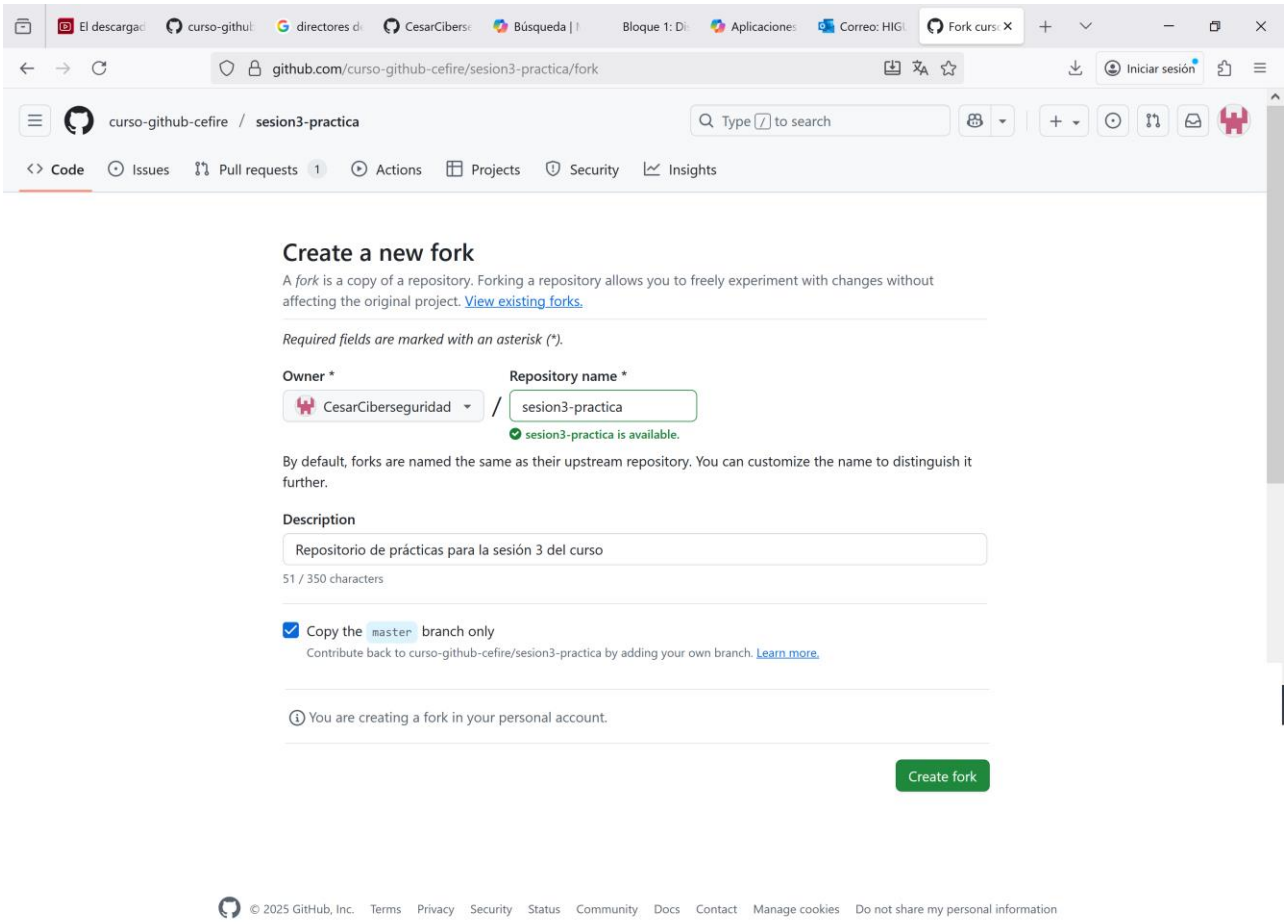


b) Actualiza la rama master local con el contenido de la rama master del repositorio remoto. Adjunta captura de pantalla del resultado del comando utilizado.



4. GitHub

1. Crea una cuenta en GitHub
2. Anade tu direccion de correo de educacion
3. Solicita un descuento para uso educativo: https://education.github.com/pack
4. Haz un fork del repositorio localizado en la siguiente url: https://github.com/curso-github-cefire/sesion3-practica. A partir de este momento todas las tareas que se indican se deben realizar en tu repositorio (el que has clonado mediante el fork).

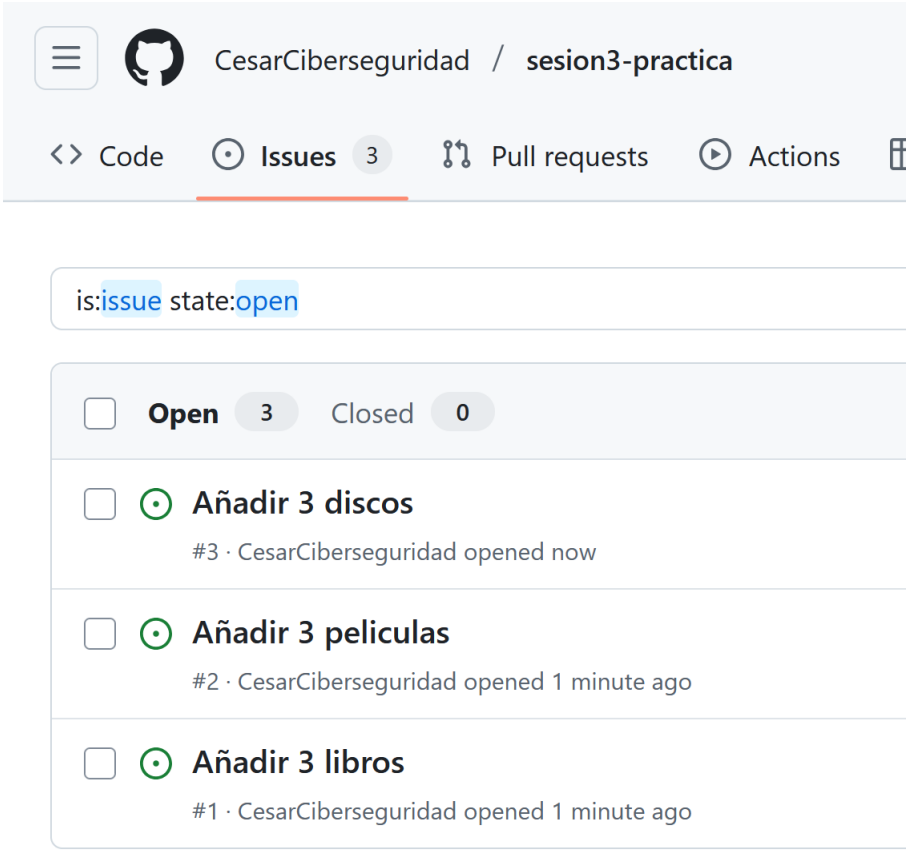


a) Realiza un primer commit para poner tu nombre y apellidos en el fichero README.md

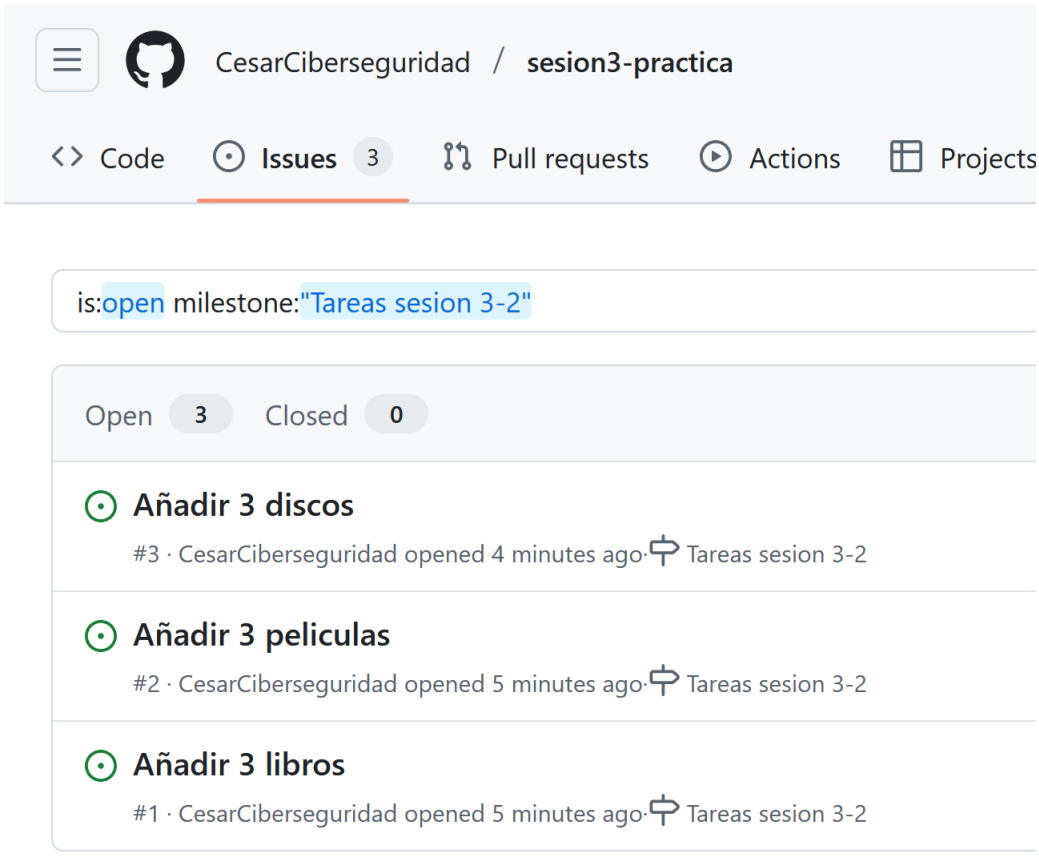
git clone https://github.com/CesarCiberseguridad/sesion3-practica.git
git add .
git commit –m “Nombre en README.md”
git push –u origin master

```
C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git push -u origi
n master
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/CesarCiberseguridad/sesion3-practica.git
   a692f8f..c4fcb08  master -> master
branch 'master' set up to track 'origin/master'.
```

b) Crea 3 issues con los siguientes titulos. Si no ves la pestana de issues, activala desde los ajustes (settings) del repositorio.
Anadir 3 libros
Anadir 3 peliculas
Anadir 3 discos



b) Crea una milestone denominada Tareas sesion 3-2 que contenga los 3 issues creados.



d) Modifica los ficheros correspondientes y realiza 3 commits para realizar cada una de las tareas que se indican en los issues. El mensaje del commit debe hacer que se cierren los issues correspondientes de manera automatica.
e) Haz una captura de pantalla de los comandos que has utilizado para hacer los commits y subir los cambios a GitHub.

git add libros.txt
git commit –m “commit libros. Close #1”
git push

```
C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git status
On branch master
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   libros.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   discos.txt
        modified:   peliculas.txt


C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git commit -m "co
mmit libros. Close #1"
[master 7272f7d] commit libros. Close #1
 1 file changed, 4 insertions(+)

C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 371 bytes | 185.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CesarCiberseguridad/sesion3-practica.git
   c4fcb08..7272f7d  master -> master
```

C:\Windows\System32\cmd.exe

```
C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   discos.txt
        modified:   peliculas.txt

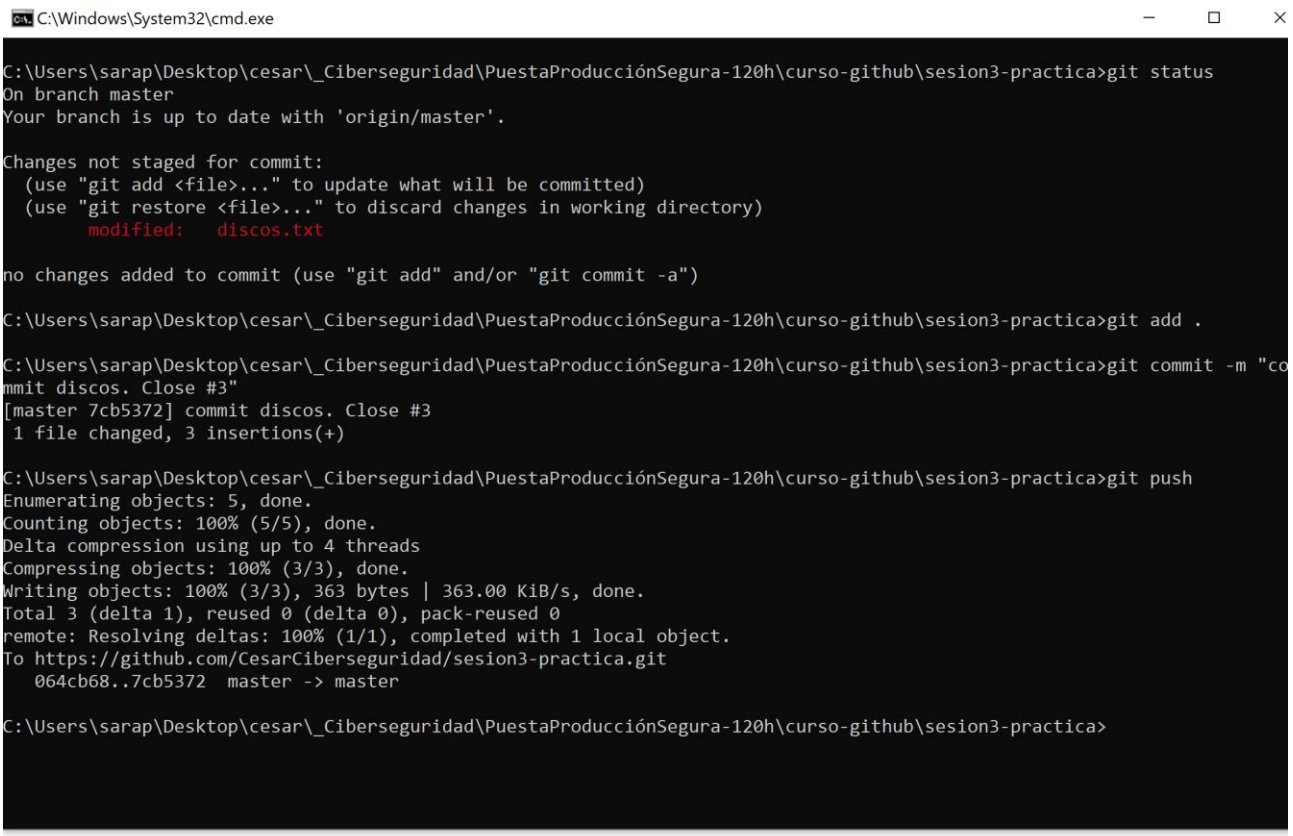
no changes added to commit (use "git add" and/or "git commit -a")

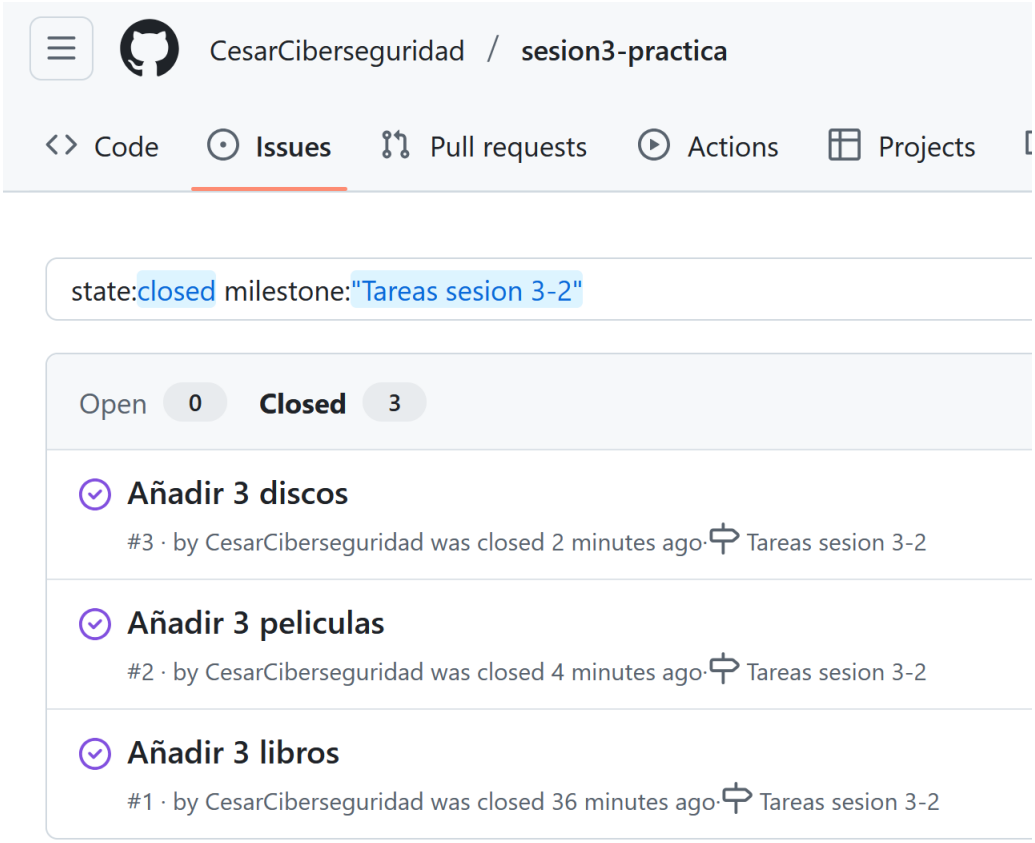
C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git add peliculas
.txt

C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git commit -m "co
mmit peliculas. Close #2"
[master 064cb68] commit peliculas. Close #2
 1 file changed, 3 insertions(+)

C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 379 bytes | 189.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CesarCiberseguridad/sesion3-practica.git
   7272f7d..064cb68  master -> master

C:\Users\sarap\Desktop\cesar\_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>
```


C:\Windows\System32\cmd.exe
C:\Users\sarap\Desktop\cesar_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git status
On branch master
Your branch is up to date with 'origin/master'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: discos.txt
no changes added to commit (use "git add" and/or "git commit -a")
C:\Users\sarap\Desktop\cesar_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git add .
C:\Users\sarap\Desktop\cesar_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git commit -m "co
mmit discos. Close #3"
[master 7cb5372] commit discos. Close #3
1 file changed, 3 insertions(+)
C:\Users\sarap\Desktop\cesar_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 363 bytes | 363.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CesarCiberseguridad/sesion3-practica.git
064cb68..7cb5372 master -> master
C:\Users\sarap\Desktop\cesar_Ciberseguridad\PuestaProducciónSegura-120h\curso-github\sesion3-practica>


CesarCiberseguridad / sesion3-practica
Code
Issues
Pull requests
Actions
Projects
state:closed milestone:"Tareas sesion 3-2"
Open 0
Closed 3
Añadir 3 discos
#3 · by CesarCiberseguridad was closed 2 minutes ago · Tareas sesion 3-2
Añadir 3 peliculas
#2 · by CesarCiberseguridad was closed 4 minutes ago · Tareas sesion 3-2
Añadir 3 libros
#1 · by CesarCiberseguridad was closed 36 minutes ago · Tareas sesion 3-2

f) Incluye las capturas de pantalla en el repositorio dentro de la carpeta capturas. Añadelas tambien al repositorio de manera que queden guardadas en tu repositorio en GitHub.

git add .
git commit –m “commit capturas”
git push

g) Realiza una pull request indicando en el mensaje que has completado la tarea.